Progress Report

Comparing LLM-Based Policy Generation and Reinforcement Learning for Embodied Control

Jarne Demoen

1 Week 1: Initial Pipeline and Baseline Evaluation

1.1 Goal of the Week

The goal of Week 1 was to create a minimal working experimental pipeline for the project. Instead of immediately starting with LLM-generated controllers, I first focused on a simple continuous-control environment. The main objective was to verify that I could create an environment, evaluate fixed controllers, train a reinforcement learning baseline, and save the results in a structured way.

1.2 Environment

The first environment used was `Pendulum-v1` from Gymnasium. I selected this environment because it has continuous observations and continuous actions. This makes it more relevant to the embodied-control setting of the project than a discrete task such as `CartPole`, while still being simple enough for debugging.

The observation contains three values:

- $\cos(\theta)$, the cosine of the pendulum angle;
- $\sin(\theta)$, the sine of the pendulum angle;
- $\dot{\theta}$, the angular velocity.

The action is one continuous torque value in the interval $[-2, 2]$. The goal is to apply torque so that the pendulum reaches and stays near the upright position. In this environment, rewards are usually negative, so values closer to zero indicate better performance.

1.3 Implemented Methods

Three methods were considered in the Week 1 report: a random controller, a manually defined controller, and a PPO baseline.

1.3.1 Random Controller

The random controller samples a torque uniformly from the valid action range at each time step. It does not use the observation and does not learn. It is included as a lower baseline to check whether other methods can do better than random actions.

1.3.2 Manual Controller

The manual controller is a fixed hand-written rule. It uses the pendulum angle and angular velocity to compute a torque. The angle is reconstructed from the observation using

$$\theta = \text{atan2}(\sin(\theta), \cos(\theta)).$$

The controller then applies a simple proportional-derivative style rule:

$$u = -2.0\theta - 0.5\dot{\theta}.$$

Here, u is the torque applied to the pendulum. The value of u is clipped to the valid torque range $[-2, 2]$. This controller is not trained; it is only a simple manually designed policy.

1.3.3 PPO Baseline

Proximal Policy Optimization, or PPO, is a policy-gradient reinforcement learning algorithm. It learns a parameterized policy $\pi_\theta(a \mid s)$, where s is the current state, a is the selected action, and θ represents the parameters of the neural network policy. During training, the agent interacts with the environment, collects trajectories, estimates which actions were better or worse than expected, and updates the policy.

The main idea behind PPO is to improve the policy without making updates that are too large. Large policy updates can make training unstable. PPO therefore compares the probability of an action under the new policy with the probability of the same action under the old policy. This probability ratio is defined as

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

PPO optimizes a clipped objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right].$$

Here, $\hat{A}_t$ is the estimated advantage and ϵ is the clipping parameter. The advantage indicates whether an action was better or worse than expected. The clipping term limits how much the new policy can deviate from the old policy during one update. This makes PPO relatively stable and easy to use compared to older policy-gradient methods.

The first reinforcement learning baseline was trained using Proximal Policy Optimization, implemented with Stable-Baselines3. PPO was included to check whether the pipeline can compare fixed controllers with a learned policy.

In the Week 1 code, PPO was trained on `Pendulum-v1` for 100,000 timesteps using mostly the default Stable-Baselines3 PPO settings. The policy architecture was `MlpPolicy`, which is also used in the reported Stable-Baselines3 RL-Zoo setup. The difference with the reported RL-Zoo result is mainly due to the training setup rather than the policy architecture. Both setups use `MlpPolicy`, but the Week 1 run used a single `gym.make` environment and mostly default PPO hyperparameters, while the RL-Zoo configuration uses tuned hyperparameters, vectorized environments, and generalized State Dependent Exploration. Exact reproduction of the RL-Zoo result would require matching the full RL-Zoo training setup, including wrappers, vectorized environments, hyperparameters, library versions, and evaluation protocol. This is not the main objective of the current project. Instead, the main purpose of PPO in this project is to provide a trained reinforcement learning baseline against which LLM-generated and iteratively refined controllers can be compared. Therefore, the most important requirement is that PPO is trained and evaluated consistently within the same experimental pipeline as the other methods.

1.4 Evaluation Setup

The random and manual controllers were evaluated on `Pendulum-v1` for 10 episodes using seed 42. The PPO model was trained for 100,000 timesteps and evaluated for 10 episodes. This means that the PPO result should be interpreted as a preliminary baseline test, not as a final comparison.

The reported metric is the mean cumulative reward over the evaluation episodes. Since rewards in `Pendulum-v1` are negative, higher values are better.

1.5 Results

Table 1 shows the Week 1 results.

Table 1: Week 1 results on `Pendulum-v1`. Higher reward is better.

Method	Episodes	Mean reward	Std. reward	Training
Random controller	10	-1248.195	271.625	No
Manual controller	10	-1180.920	234.175	No
PPO baseline	10	-1075.093	206.479	Yes

The manual controller performed better than the random controller. The improvement in mean reward shows that the manually defined rule uses some useful information from the state. However, the improvement is limited.

The PPO baseline obtained a mean reward of -1075.093. This is better than both fixed controllers, but the difference is not as large as expected for a well-trained PPO agent on `Pendulum-v1`. The current ranking is therefore

$$\text{PPO} > \text{Manual controller} > \text{Random controller}.$$

1.6 Comparison with Reported PPO Results

The Stable-Baselines3 RL-Zoo model for PPO on `Pendulum-v1` reports a reward of approximately -230.42 ± 142.54. My PPO result was -1075.093 ± 206.479, which is much worse.

This difference does not necessarily mean that the implementation is broken, but it does mean that my PPO result is not directly comparable with the reported RL-Zoo result. The main reason is that the Week 1 PPO code did not reproduce the RL-Zoo setup. It used a simple `gym.make` environment, default Stable-Baselines3 PPO hyperparameters, one environment, and only 10 evaluation episodes. The RL-Zoo result, on the other hand, uses a tuned training configuration for `Pendulum-v1`, including specific hyperparameters, vectorized environments, and generalized State Dependent Exploration.

For this reason, the literature comparison should be interpreted carefully. The correct conclusion is not that PPO performs poorly in general, but that the current Week 1 PPO setup is only a first pipeline test. It confirms that PPO can be trained and evaluated in the project code, but it does not reproduce the expected Stable-Baselines3 RL-Zoo performance.

1.7 Discussion

The Week 1 pipeline provides three useful reference points. The random controller gives a lower baseline. The manual controller shows what can be achieved with a simple hand-written control rule. PPO shows that a learned policy can already improve over both fixed controllers, even though the current PPO setup is still weak compared to the reported RL-Zoo result.

This is useful for the next phase of the project. An LLM-generated controller should first be compared against the random and manual controllers. If it performs close to random, it is not useful. If it performs close to or better than the manual controller, it shows that the LLM can generate a meaningful control rule. PPO should eventually serve as a stronger trained baseline.

1.8 Limitations

The Week 1 results are preliminary. The main limitations are:

- only one environment was used;
- only one seed was used;
- the PPO setup did not match the RL-Zoo configuration;
- PPO was evaluated over only 10 episodes;
- the fixed controllers and PPO did not use exactly the same evaluation protocol;
- the PPO logging did not initially include all reproducibility details;
- SAC and DDPG were not included in the Week 1 report;
- no LLM-generated controller has been evaluated yet.

1.9 Conclusion

Week 1 established a working experimental setup. I can now create a Gymnasium environment, evaluate fixed controllers, train a PPO baseline, and save results. The manual controller improved over the random controller, and PPO improved over both fixed controllers.

However, the PPO result is still far from the reported Stable-Baselines3 RL-Zoo result for the same environment. This is expected because the Week 1 PPO code did not reproduce the RL-Zoo setup. The next step is to improve the PPO baseline by using the correct RL-Zoo-style hyperparameters, logging all evaluation details, and evaluating PPO with the same protocol as the other methods.

1.10 Next Steps

The next steps are:

1. Fix PPO logging so that the CSV includes the number of evaluation episodes, hyperparameters, number of environments, and model path.
2. Re-run PPO using the Stable-Baselines3 RL-Zoo configuration as closely as possible.
3. Evaluate PPO with the same number of episodes as the fixed controllers.
4. Train SAC and DDPG as additional RL baselines in a later week.
5. Generate a first LLM-based controller for `Pendulum-v1`.
6. Evaluate the LLM-generated controller against the random, manual, and PPO baselines.

References

[1] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347, 2017.

[2] Stable-Baselines3. PPO Agent Playing `Pendulum-v1`: Results JSON. Hugging Face, 2024. Available at: https://huggingface.co/sb3/ppo-Pendulum-v1/blob/main/results.json.

[3] Demoen, J. Comparing LLM-Based Policy Generation and Reinforcement Learning for Embodied Control. Estudo Dirigido Project Proposal, 2026.